

Intelligent Autonomous Navigation System

Technical Report

Name: Obaidullah Habibi
Matriculation number: 807634
Course: AINB / AI Programming, Summer Semester 2026
Supervisor: Prof. Dr. Isabel Hübener

1. Problem Description

My task was to make a mobile robot travel from a fixed start point to a goal in a partly known map. The map contains walls that are known before the run, but it can also contain moving circular obstacles. These moving objects are not included in the global map, so the robot has to notice and avoid them while it is driving.

A grid path alone is not enough for this problem. The robot has a maximum speed, turning limit and acceleration limit. It cannot instantly jump from one grid cell to the next or change direction by 90 degrees. For that reason I used two planning levels. The global planner finds a route through the complete static map. The local planner runs at every control step and decides which speed and turn rate are possible at that moment. I tested the system on four maps. The simple map is a basic check with two blocks. The maze has narrow corridors. The trap contains a U-shaped obstacle, which is useful because a local-only controller can enter it and get stuck. The dynamic map adds moving obstacles and tests whether the local controller can change the actual path during the run.

2. System Architecture

I separated the program into modules instead of putting the whole navigation loop in one file. The main parts are the environment, robot model, global planner, local planner, navigator, visualisation and experiment scripts. This made debugging easier

because I could run the search algorithms without the robot, and I could plot one DWA decision without running a complete scenario.

The navigator is the connection between the two planning levels. It receives an A* route, selects a look-ahead point on that route and gives this point to the local planner. The local planner returns a linear velocity and an angular velocity. If the robot stops making progress, the navigator asks for a new global route from the current cell.

3. Environment Representation

The environment is a 50×50 occupancy grid. A cell value of 1 means occupied and 0 means free. Static rectangles are written into the grid before planning. Dynamic obstacles are stored separately with position, velocity and radius because their positions change during execution.

For clearance checks I use a distance transform. Each free cell stores its Euclidean distance to the nearest wall. The DWA planner evaluates many predicted points in one step, so reading one value from this array is much cheaper than checking the distance to every rectangle again and again.

One implementation change was important here. An earlier version used a grid/BFS-style distance value. It was fast, but diagonal gaps were not measured as true straight-line distance. I replaced it with a Euclidean distance transform so that the clearance values used in the score and in the results mean the same thing.

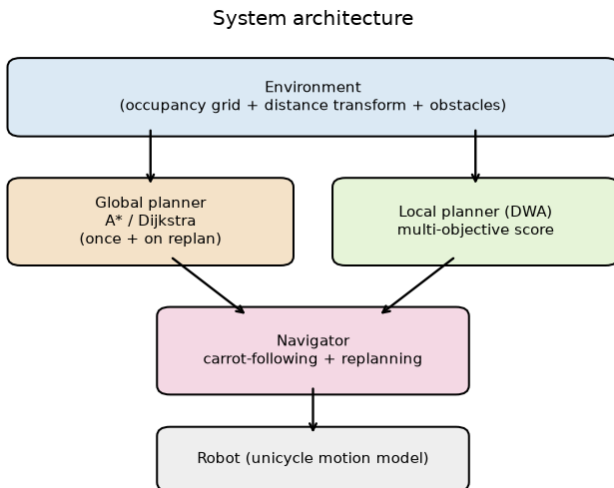


Figure. Modules and data flow used in the program.

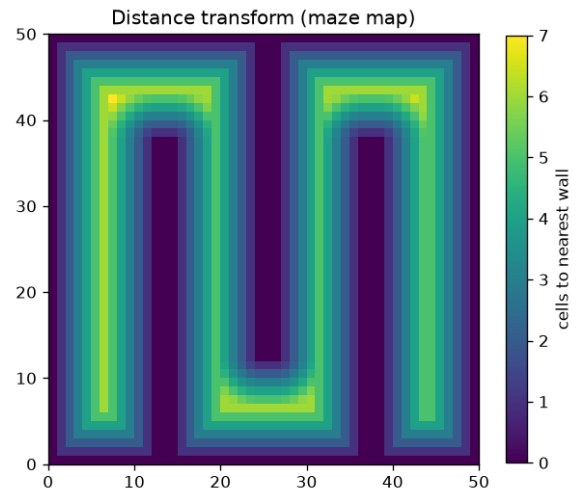


Figure. Distance transform for the maze. Bright cells are farther from a wall.

4. Global Planning Method

I implemented both Dijkstra and A* on an eight-connected grid. Python's `heapq` is used for the priority queue, but neighbour generation, cost updates, the closed/open handling and path reconstruction are in my own code.

Horizontal and vertical moves cost 1. Diagonal moves cost $\sqrt{2}$. A diagonal is rejected when it would cut between two obstacle corners. Dijkstra orders nodes only by the cost already travelled, $g(n)$. A* uses

$$f(n) = g(n) + h(n)$$

with the octile heuristic

$$h(n) = d_x + d_y + (\sqrt{2} - 2) \min(d_x, d_y).$$

This heuristic matches free movement on an eight-connected grid and does not overestimate the remaining cost. In my tests A* and Dijkstra therefore returned the same path cost, but A* usually expanded fewer cells.

Map	Dijkstra	A*	Cost
Simple	1989	628	64.53
Maze	1592	1139	115.60
Trap	2036	1101	68.63
Dynamic	2019	588	63.36

The maze is the case where A* helps least. The heuristic points roughly toward the goal, but the walls force the search to move

sideways and sometimes away from the goal. I recorded expanded nodes and path cost for this comparison. I did not include wall-clock timing, so the result should be read as a search-effort comparison rather than a complete performance benchmark.

Global planning is also needed for the trap map. To escape a U-shape, the robot may need to move away from the goal for a while. A local controller that only rewards immediate goal progress is likely to choose the wrong side of this decision.

5. Local Planning with DWA

The robot uses a unicycle state $(x, y, \theta, v, \omega)$. For every control step, the Dynamic Window Approach first creates the set of velocity commands that can be reached under the acceleration limits. Each candidate (v, ω) is simulated for a short horizon using

$$\theta' = \theta + \omega \Delta t, \quad x' = x + v \cos(\theta') \Delta t, \quad y' = y + v \sin(\theta') \Delta t.$$

A rollout is removed if any predicted point hits a wall or a moving obstacle. The remaining rollouts are scored, and the best command is applied for one step. Then the whole calculation is repeated.

The local target is not always the final goal. I use a carrot point several cells ahead on the A* path. This avoids forcing the robot to follow every sharp grid corner, while still keeping the motion connected to the global route.

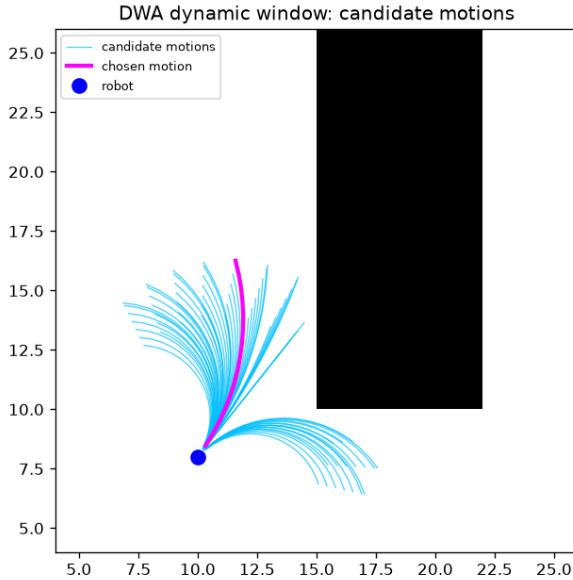


Figure. One DWA step. Blue curves are feasible rollouts and magenta is the selected motion.

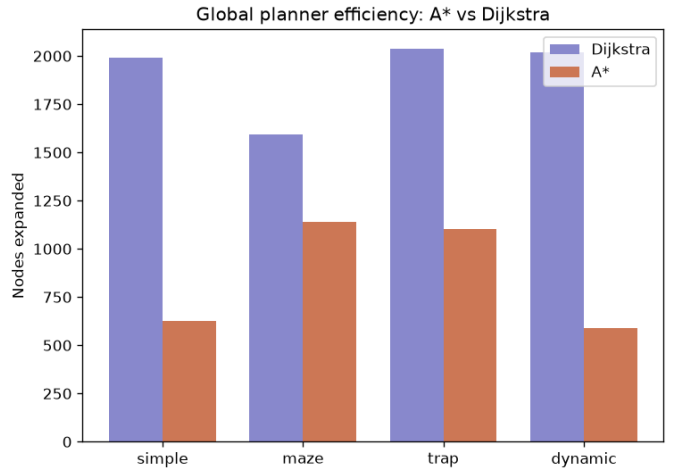


Figure. A* expands fewer nodes on all four maps, with the smallest advantage in the maze.

6. Optional Extension: Multi-Objective Score

I chose Option D from the project description. The basic DWA score uses heading, obstacle distance and velocity. I added two more terms for clearance over the full rollout and smooth steering:

$$G = \alpha H + \beta O + \gamma V + \delta C + \epsilon S.$$

H measures final heading toward the carrot, O uses obstacle distance near the end of the rollout, V rewards forward speed, C is average clearance along the rollout and S rewards small and gradual steering changes.

The five values have different units, so I normalise them across the candidate set before applying the weights. I also clip the clearance reward above a safe level. Without clipping, the controller can prefer an empty area forever instead of continuing toward the goal.

I used four weight presets:

Preset	α	β	γ	δ	ϵ
Aggressive	1.4	0.5	1.4	0.2	0.2
Balanced	1.0	1.0	0.7	0.6	0.4
Smooth	1.1	0.9	0.7	0.5	0.9
Cautious	0.8	1.6	0.4	1.5	0.6

The names describe the intention, not a guarantee. For example, the smooth preset did not always make the shortest or cleanest-looking route. In the dynamic test it made a large loop and needed one replan. This happened because avoiding a sudden steering correction can be more expensive later.

7. Experimental Method

All runs used the same map size, start at (5,5) and goal near (44,44). For global search I recorded expanded nodes and path

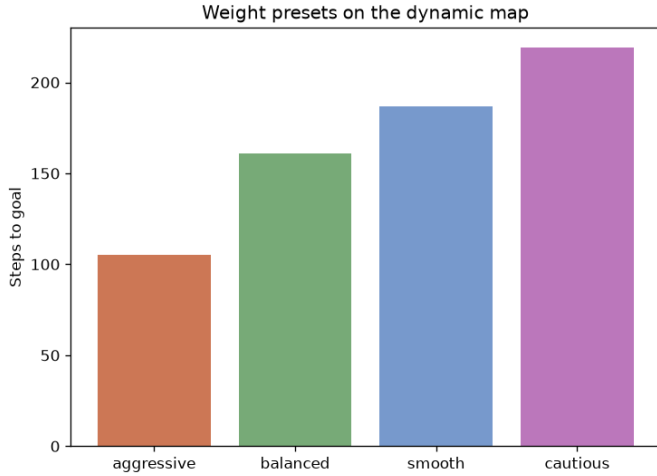


Figure. Steps needed by each preset on the dynamic map.

cost. For complete navigation I recorded control steps, travelled path length, number of replans, minimum clearance and average clearance. A run ended when the robot entered the goal region or reached the step limit.

The figures were generated from the program output. I used them to check that the written explanation matched the actual route. This was especially useful for the dynamic map, where the orange robot path should differ from the dashed A* path because moving obstacles are not part of the static grid.

8. Weight Study Results

The presets were run on the dynamic map. The measured values were:

Preset	Steps	Length	Replans	Min	Avg
Aggressive	105	60.7	0	1.00	3.31
Balanced	161	91.4	0	2.04	4.24
Smooth	187	106.1	1	1.34	4.05
Cautious	219	70.5	0	2.89	3.94

Aggressive was fastest and had the shortest path, but it also came closest to an obstacle. Cautious had the best minimum clearance. Balanced had the largest average clearance, although its worst moment was still closer than cautious. This is why I think minimum clearance is the more useful safety value in this experiment.

The cautious run also shows that steps and path length are not the same measure. It took many steps but travelled only 70.5 cells. The robot was moving more slowly and keeping space, rather than constantly making a very long route.

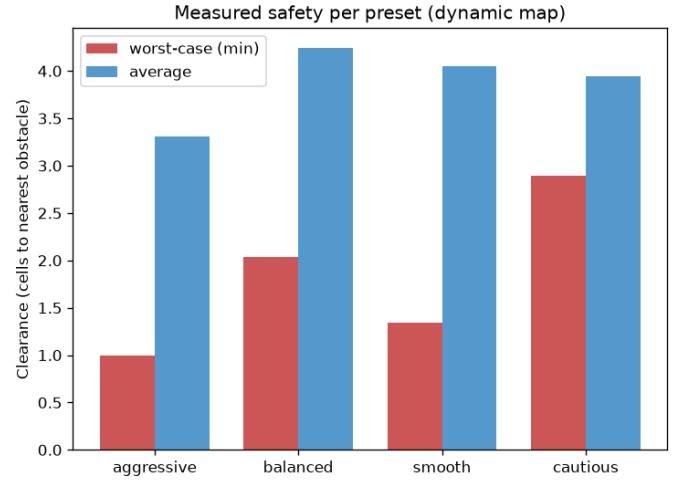


Figure. Minimum and average clearance. Cautious has the best worst-case value.

9. Interpreting the Presets

No preset was best in every measurement. Aggressive is the obvious choice when time and distance are the main priorities, but its minimum clearance was only 1.00 cell. Cautious increased this to 2.89 cells, but it needed 219 steps. Smooth produced the longest route and one replan. Balanced stayed between the extremes and completed the dynamic run in 161 steps.

The path plot explains some of the numbers better than the table. Aggressive makes small avoidance actions and then returns to a direct line. Cautious makes a larger early detour. Smooth creates broad curves and loops because it avoids quick changes in turning rate.

The average clearance values are fairly close, even when the minimum values are different. This is partly caused by clipping the safety reward. In open space, extra distance above the safe level is not rewarded. The presets mainly separate when the robot has to pass near an obstacle.

For a real application I would not choose the weights only by looking at one run. A warehouse robot near people would need repeated safety tests and probably a stricter collision margin. A robot in an empty test area could use a faster setting. In this

project the weights are selected manually; the system does not learn them.

10. What the Trade-Off Plot Shows

The scatter plot uses steps to goal on the horizontal axis and path length on the vertical axis. Aggressive is low on both axes. Smooth is high on path length, while cautious has many steps but a shorter geometric route than balanced or smooth.

This plot should be read together with the clearance chart. A point that looks good for time and distance may still be the least safe. The experiment therefore supports the reason for using a multi-objective score: one number is not enough to describe the behaviour.

11. A Result That Surprised Me

I expected the smooth preset to look better in the dynamic scene because it has the highest smoothness weight. Instead, it made a large loop and produced the longest path. The controller was doing what the score asked, but the local choice was not globally efficient. This is a useful failure case because it shows that adding another objective does not automatically improve the final route.

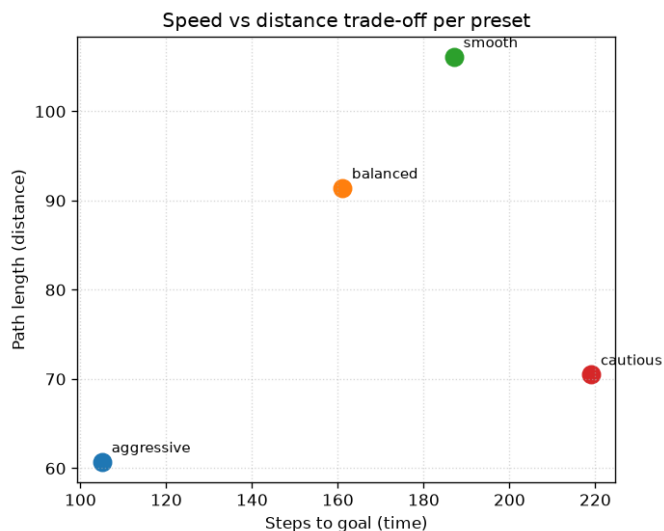


Figure. Time and path-length trade-off. Safety is not shown on this plot, so it must be checked separately.

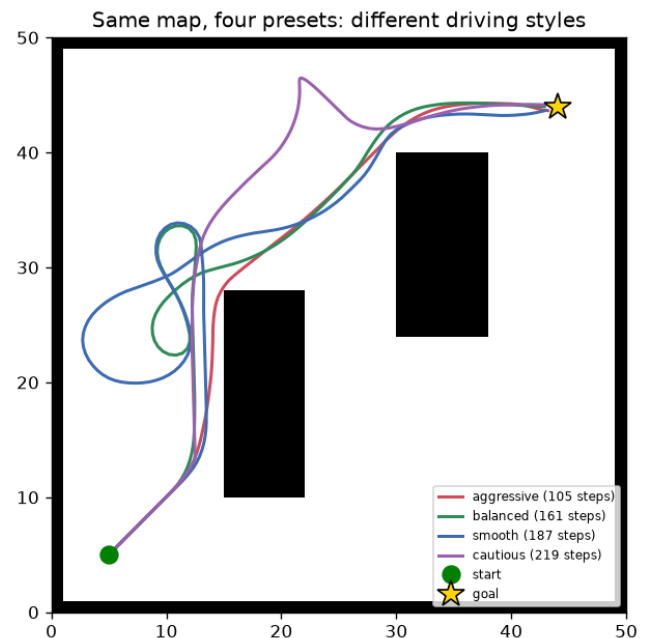


Figure. Same map and goal, but different paths produced only by changing the DWA weights.

12. Scenario Evaluation

The complete system reached the goal in all four scenarios.

Simple: 111 steps and no replan. The robot path is smoother than the grid path because the DWA controller follows a look-ahead point and uses continuous motion. I do not treat this difference as an error; forcing the robot through every grid corner would create unnecessary turns.

Maze: 216 steps and one replan. This was the most difficult static case. A* selected the correct corridor sequence, but rounded local turns caused the robot to lose progress in a narrow part. Replanning from the current cell gave the navigator a new carrot sequence and the run continued.

Trap: 123 steps and no replan. The important part is the route shape. The global path goes around the outside of the U-shaped obstacle. A local-only controller can enter the opening because the goal appears to be closer there, even though the route later

becomes blocked.

Dynamic: 161 steps and no replan with the balanced preset. The dashed A* route ignores the moving circles. The robot path bends and loops when an obstacle crosses the local horizon. The final path is therefore a combination of long-range guidance and short-range reaction.

Across the four maps, the A* and robot paths are close in open space and different near corners or moving obstacles. The grid route is guidance, not an exact motion command.

13. Replanning Rule

I did not replan after every small deviation. The navigator requests a new global route only when progress is too small or the local planner remains blocked. This avoids repeatedly changing the global path because of normal local steering. Three scenarios completed with the original path, and only the maze required a replan.

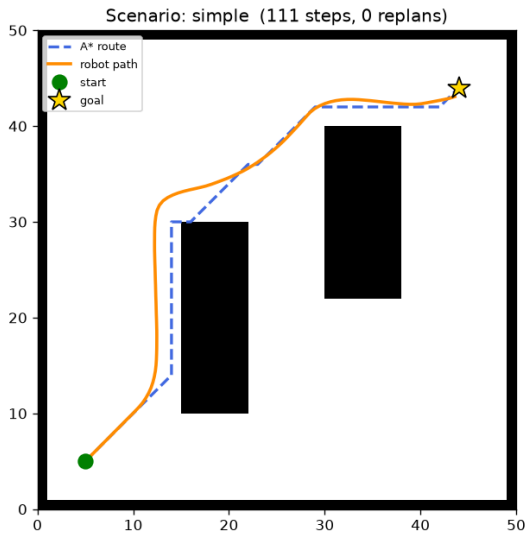


Figure. Simple: 111 steps, 0 replans.

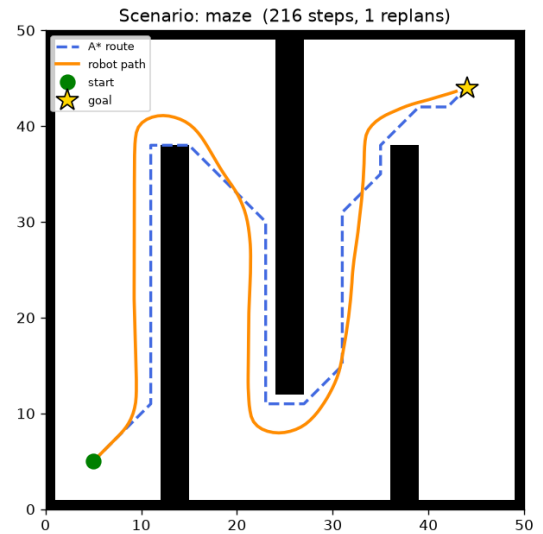


Figure. Maze: 216 steps, 1 replan.

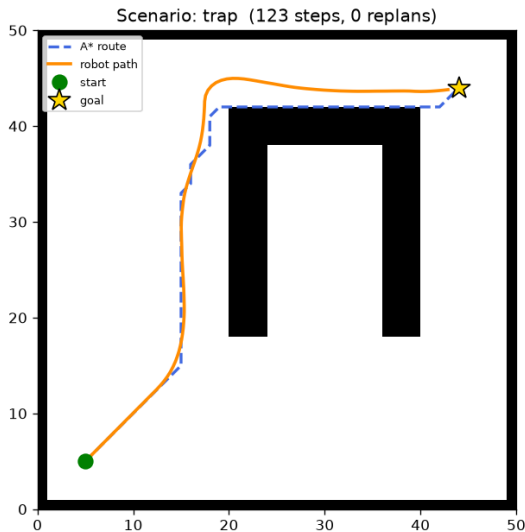


Figure. Trap: 123 steps, 0 replans.

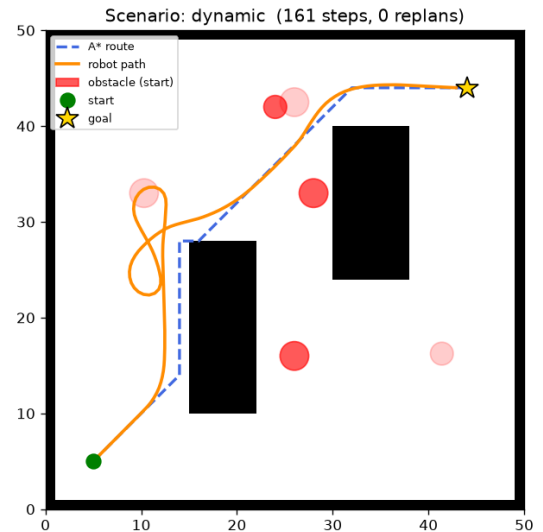


Figure. Dynamic: 161 steps, 0 replans.

14. Discussion and Failure Cases

The strongest part of the system is that the two planners solve different problems. A* can see the full static map and avoid dead ends. DWA can respect acceleration and turning limits and can react to obstacles that were not known when the global path was created.

There are still several limitations:

- The DWA collision check uses the current position of a moving obstacle. It does not predict where the obstacle will be later in the rollout. A fast obstacle moving toward the robot can therefore reduce the reaction time.
- The grid resolution limits path precision. A finer grid gives more detail but increases the search space and memory use. A coarse grid can also make a narrow gap disappear.
- The distance transform is computed for static walls. Moving obstacles are checked separately. If the map itself changed permanently, the distance field would need to be recomputed.
- The score is sensitive to its weights and horizon. Too much speed reduces clearance. Too much smoothness can create loops. Too much safety can make the robot hesitate.
- If every sampled command collides, the robot has to stop. Replanning can help when another global route exists, but it cannot solve a completely blocked corridor.

A practical improvement would be to predict the motion of dynamic obstacles and compare each robot rollout with their predicted positions. Other improvements are obstacle inflation based on speed, adaptive sampling near good DWA commands and automatic tuning of the weights from repeated runs.

15. Reflection: What Makes the System Intelligent?

I do not think one part of the program is intelligent by itself. The behaviour comes from how the parts are connected. Search gives long-range knowledge of the static map. The DWA score compares possible actions under several constraints. The occupancy grid and distance transform make those calculations

practical. Replanning allows the controller to recover when the old route is no longer useful.

Global planning alone would produce a route that ignores the robot's motion limits and moving obstacles. Local planning alone reacts quickly, but it has no understanding of the full obstacle structure and can enter the trap. The combined system handles both cases better.

The weight presets also show a limited form of reasoning about priorities. The controller can prefer speed, clearance or smoothness, but it does not learn these priorities. They were chosen manually, so this is multi-objective optimisation rather than machine learning.

16. Conclusion

The project produced a working Python navigation framework with manually implemented Dijkstra and A*, a unicycle robot model, DWA local control and a five-term multi-objective extension. A* kept the same optimal path cost as Dijkstra while expanding fewer nodes. The four presets produced measurable differences in time, path length and safety. The robot reached the goal in the simple, maze, trap and dynamic scenarios, with one recovery by replanning in the maze.

The main lesson for me was that small implementation choices matter. The Euclidean distance field made the safety values consistent, the carrot point connected a grid route to continuous motion, and the replan condition prevented unnecessary global searches. The system is not perfect, but its decisions can be explained from the map, the global path and the local score.

Reproduction Commands

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python main.py
python main.py --scenario trap
python main.py --experiments
python -m navigation_system.figures
```